

第 1 頁：封面

大家好，我今天要報告的題目是神經網路中風預測分析。這份期末報告是延續期中的中風預測分析，期中主要使用傳統機器學習模型，期末則加入神經網路，使用 `MLPClassifier` 作為延伸主軸。

第 2 頁：研究延伸

這一頁說明期中和期末的差異。期中我已經完成資料前處理，也比較了邏輯斯迴歸 (`Logistic Regression`)、隨機森林 (`Random Forest`) 和 `KNN`，並使用 `class_weight` 和 `SMOTE` 處理類別不平衡。

期末的重點是加入神經網路，測試 `MLP` 能不能學到傳統模型較難捕捉的非線性關係。這次使用 `MLPClassifier` 作為主模型，搭配 `GridSearchCV` 搜尋參數，最後用 `Out-of-Fold, OOF` 和 `F2` 分數 (`F2-score`)，選擇較適合醫療預警的分類門檻。

第 3 頁：資料集

這份資料集共有 5,110 筆資料，目標是預測是否中風。資料最大的問題是中風案例很少，只有 249 筆，大約占 4.87%。

所以這份資料不能只看準確率 (`Accuracy`)，因為模型如果全部預測成沒有中風，準確率也可能很高，但醫療上沒有意義。因此這次更重視 `Recall`、`Precision`、`F1` 分數、`F2` 分數、`PR-AUC` 和混淆矩陣。

第 4 頁：探索性視覺化

從視覺化可以看到，中風案例比較集中在高年齡區間，平均血糖較高時，中風風險也比較明顯。不過單一特徵不能直接決定中風，比較合理的解釋是，年齡、血糖、`BMI`、高血壓和生活型態等多個弱訊號共同累積，最後形成較高風險。

第 5 頁：缺失值與相關性

資料中 `BMI` 有 201 筆缺失值，這次我把補值放進 `Pipeline` 裡面，讓模型只用訓練資料計算 `BMI` 的中位數，再拿來補缺失值。這樣測試資料不會提前被參考，可以降低資料洩漏風險。

相關性分析中，年齡和平均血糖與中風有較明顯的正向關係，所以我把它們當作風險訊號。

第 6 頁：編碼與標準化

這一頁是前處理流程。數值欄位使用中位數 補值和標準化工具，類別欄位使用最常見值 補值和 One-Hot Encoding。

我使用欄位轉換器加上 Pipeline，把補值、編碼、標準化和模型放在一起。這樣在交叉驗證時，每一折都只會從訓練資料學習前處理方式，較能避免資料洩漏。

第 7 頁：神經網路流程

整體流程是先把資料分層 切分成訓練集和測試集。訓練集有 4,088 筆，測試集有 1,022 筆。接著用 Pipeline 完成前處理，再用 MLPClassifier 做模型訓練。

參數搜尋使用 3-fold GridSearchCV，評分指標不用準確率，是用 PR-AUC，因為它較適合這種不平衡資料。最後再用訓練集的 OOF 結果，選擇 F2 分數最高的分類門檻。測試集只在最後一次評估時使用。

第 8 頁：神經網路設計

這次選擇 MLP，是因為資料是表格型資料，不是影像或時間序列，所以不採用 CNN、RNN 或 LSTM

MLP 適合這種欄位型資料，它可以學習欄位之間的非線性組合。如年齡、血糖、高血壓和 BMI。這次用 PR-AUC 當調參目標，用 F2 分數當門檻選擇目標。

PR-AUC 用來看風險排序能力，F2 則比較重視召回率，符合醫療預警中少漏判的需求。

第 9 頁：神經網路超參數調整

這一頁是 MLP GridSearchCV 調參結果。我測試了三種架構：第一種是一層 32 個神經元、第二種是一層 64 個神經元，第三種是兩層隱藏層，第一層 64 個神經元、第二層 32 個神經元。

激活函數比較 ReLU 和 tanh，也測試不同的正則化參數 alpha 和學習率。最後最佳 MLP 是一層 64 個神經元，激活函數使用 ReLU，alpha 是 0.0001，學習率是 0.001，交叉驗證的 PR-AUC 是 0.1639。

第 10 頁：決策門檻

一般分類模型會用 0.5 當門檻，但在中風預測中，0.5 可能太保守，會漏掉較多真正中風的人。所以這次我用 OOF 和 F2 分數來選門檻，最後最佳門檻是 0.245。

使用預設 0.5 時，偽陰性 (FN) 是 33，只抓到 17 位中風者；調整到 0.245 後，偽陰性 (FN) 降到 11，抓到 39 位中風者。

代表模型更符合醫療預警中少漏判的需求。

第 11 頁：門檻選擇

我使用訓練集的 OOF 預測機率來找最佳門檻。因為若用測試集挑門檻，分數會被高估，等於模型偷看考題。

門檻固定後，才拿測試集做評估。最後測試集召回率達到 0.78，也就是 50 位中風者中抓到 39 位。雖然 偽陽性 (FP) 增加，但在醫療預警中，先不要漏掉高風險個案比較重要。

門檻用訓練集的 OOF 選，測試集只負責最後驗證。

第 12 頁：神經網路混淆矩陣

調整門檻後，TP (真正例) 是 39，成功抓到 39 位中風個案。偽陰性 (FN) 是 11，漏判人數從原本 33 降低到 11。偽陽性 (FP) 是 215，誤警示增加。

這裡的重點不是追求最高準確率，而是降低漏判風險。因為在中風預測中，FN 代表真正中風的人被模型判斷成沒有中風，是嚴重的錯誤。

第 13 頁：結果分析

這一頁是比較 MLP 和 LR benchmark。

MLP 的 PR-AUC 是 0.274，LR benchmark 則是 0.252，代表 MLP 在風險排序能力上有進步，較能把真正高風險的人排在前面。

不過 LR benchmark 的 Recall 是 0.800，F2 是 0.445；而 MLP 的 Recall 是 0.780，F2 是 0.430。所以不能說神經網路全面勝出，較合理的結論是：MLP 在風險排序上有改善，但分類效果還有調整空間。

第 14 頁：期中與期末比較

這一頁是期中和期末的比較。期中的模型是 **Balanced LR**，是加入 `class_weight` 的邏輯迴歸，FP 是 251，FN 是 10，召回率是 0.80。期末的 **MLP 神經網路**，FP 是 215，FN 是 11，召回率是 0.78。可以看到，**MLP** 的 FN 只多 1 位，但 FP 比期中的 **Balanced LR** 少一些。所以期末不是推翻期中，而是延伸到神經網路，觀察 **MLP** 是否能改善風險排序和錯誤警示。

第 15 頁：神經網路結果判讀

MLP 的 **PR-AUC** 高於 **LR benchmark**，代表它在排序高風險個案時有進步。**MLP** 的召回率是 0.78，也就是 50 位中風者抓到 39 位。

我認為 **MLP** 有學到一些特徵交互作用，也就是多個欄位組合後的風險訊號。不過因為中風樣本很少，神經網路仍然會受到類別不平衡影響。未來可以加入機率校準、成本敏感訓練，或更多臨床資料來改善模型。

第 16 頁：結論

這裡總結四個結論

第一，**MLP** 適合這份表格型資料，能學習非線性特徵組合。

第二，經過 **GridSearchCV** 和 **OOF F2** 門檻調整後，神經網路在測試集的 **Recall** 達到 0.78。

第三，**MLP** 的 **PR-AUC** 高於 **LR benchmark**，代表風險排序能力有提升。

第四，不過 **LR benchmark** 在 **Recall** 和 **F2** 上仍略高，所以神經網路還有優化空間。因此這個模型較適合作為初步預警工具，找出高風險個案，再搭配醫師檢查確認。

我的報告到這邊，謝謝大家。

Pipeline 可以理解成一條資料處理管線:

它是把缺失值補值、類別欄位編碼、數值欄位標準化，以及最後的 MLP 模型訓練，全部串成一個固定流程。

這樣做的好處是，在交叉驗證時，每一折都只會用訓練資料去學習補值和標準化方式，不會提前看到測試資料，所以可以降低資料洩漏的風險。

我理解成，原本 **train/test** 只是一層切分，但這次因為使用 **3-fold GridSearchCV** 和 **OOF**，所以訓練集內部還會再切成三折。**Pipeline** 的作用就是讓每一折都重新執行前處理，而且只用當下訓練折學習中位數、標準化參數和編碼方式，再套用到驗證折，避免驗證折的資訊提前被用到。

F2 是什麼？為什麼不用 F1？

F1 是 Precision 和 Recall 的平衡。

F2 則比 F1 更重視 Recall。

因為中風預測比較怕漏判，所以我不用 F1，而是用 F2。F2 會把 Recall 的重要性看成 Precision 的 4 倍，比較符合醫療預警中少漏判的需求。

F1 : Precision : Recall = 1 : 1

F2 : Precision : Recall = 1 : 4

OOF 是什麼？

OOF 是離群折預測。

簡單說，就是把訓練資料再切成幾份，每次用其中幾份訓練，剩下一份驗證。

我用 3-fold 的方式產生訓練集的 OOF 預測機率。每一輪用其中兩份訓練，剩下一份驗證並產生預測機率。三輪結束後，訓練集每一筆資料都有一個比較公平的中風風險分數。接著我用這些 OOF 分數去測試不同分類門檻，最後選出 F2 分數最高的門檻 0.245。

LR benchmark 是什麼？

LR benchmark 是邏輯斯迴歸基準模型。

我用它當作對照組，來比較 MLP 神經網路是否有改善。

Balanced LR 是什麼？

Balanced LR 是平衡 LR，也就是加入類別權重的邏輯斯迴歸。
因為中風樣本很少，所以用 `class_weight` 讓模型在訓練時更重視中風類別。

`class_weight` 跟 SMOTE 差在哪？

`class_weight` 是改變模型訓練時的權重，讓少數類別錯誤的代價變高。
SMOTE 是產生新的少數類別樣本，讓資料比較平衡。

資料處理的部分

我先移除不需要的 `id` 欄位，接著處理 BMI 缺失值。BMI 缺失值我使用中位數補值，因為中位數比較不容易受到極端值影響。

類別欄位像性別、工作類型、吸菸狀態，我使用獨熱編碼轉成 0 和 1。數值欄位像年齡、平均血糖和 BMI，則使用標準化工具，讓不同欄位的尺度比較一致。最後我把補值、編碼、標準化這些前處理步驟，和 MLP 模型一起放進資料處理管線 (Pipeline) 裡。這樣在交叉驗證時，每一折都只會用訓練資料學習前處理方式，可以降低資料洩漏風險。

期中的 ML 能不能用

可以，Pipeline 和 GridSearchCV 不是 MLP 專用，傳統機器學習也可以用。只是如果要用 OOF F2 選門檻，就要看模型的機率輸出穩不穩。邏輯斯迴歸和隨機森林比較適合，因為它們可以輸出比較有意義的機率；KNN 雖然也可以輸出機率，但它是看鄰居比例，所以會比較粗糙，拿來做門檻調整時要小心。